

COMS4995W30

Applied Machine Learning

Dr. Spencer W. Luo

Columbia University | Fall 2026



Generative vs. Discriminative Approaches: Naive Bayes vs. Linear Regression

Week of AI - Model Edition



Generating is Now Almost Free

 News: Epoch AI reports the cost of a fixed level of AI performance has fallen ~47% per quarter since 2023, roughly 13x a year. Text, code and images are close to free to produce.

Class Connection:

- LLMs are Generative models: they learn how data is produced and can write anything. That skill is becoming a commodity.
- The scarce skill is deciding: is this output correct, safe, worth paying for? That is a Discriminative problem (a boundary), and it is where linear models still earn their keep.

Week of AI - Model Edition



Agents at Bot Scale & The Missing Verifier

 News: Meta's Muse and other agents were caught querying restaurant reservation systems “at bot scale” this week; platforms answered with account bans. A separate study found paired agents rubber-stamp each other's work (66% acceptance) once verification gets expensive.

Class Connection:

- Bot vs. human is a classification with a <50 ms budget: exactly the regime where Logistic Regression / Naive Bayes beat a Transformer.
- Every agent loop = Generate (plan) $\rightarrow$ Discriminate (check) $\rightarrow$ Execute. Skip the cheap discriminator and the generator runs wild.

Agenda



- Motivation
- Naive Bayes (Generative)
- Linear Regression (Discriminative)
- Case Study



G vs. D: The Big Picture

- Generative approach (e.g., Naive Bayes)
 - Learns how the data is generated
 - Like: “learning the recipe 🍳”
- Discriminative approach (e.g., Linear Regression)
 - Directly learns the mapping
 - Like: “tasting and giving a score 🍴🍽️”
- Same goal → **make predictions**
 - Yet 2 very different philosophies 📖

Generative Philosophy



Learns the story of how data is produced (**joint distribution**)

- Examples:
 - Naive Bayes
 - assumes words are independent
 - builds probability recipe for each class
 - Language models
 - predict next word by modeling sentence generation



Generative Philosophy

✓ Strengths of Naive Bayes

-  Performs well on small datasets (low variance, simple structure)
-  Strong baseline - often hard to beat with simple methods

✗ Limitations of Naive Bayes

-  Naive assumption: features are conditionally independent
-  Fails if features are correlated (e.g., “New” + “York”)

Discriminative Philosophy



Learns the **boundary** or function that best predicts outcomes

- Examples:
 - Linear Regression (fit a line directly to minimize error)
 - Logistic Regression, SVM, modern neural networks



Discriminative Philosophy

Strengths of Discriminative Models

-  Directly optimize prediction accuracy - learn $p(y|x)$
-  Often higher accuracy than generative models (fewer assumptions)

Limitations of Discriminative Models

-  Cannot model data generation $p(x,y)$
-  Need more data than generative models to perform well

Same Goal, Different Roads.



Why This Matters?



Different learning philosophies → different trade-offs

- Generative = Once we need to understand data generation
- Discriminative = When prediction accuracy is the only goal

Mock Interview



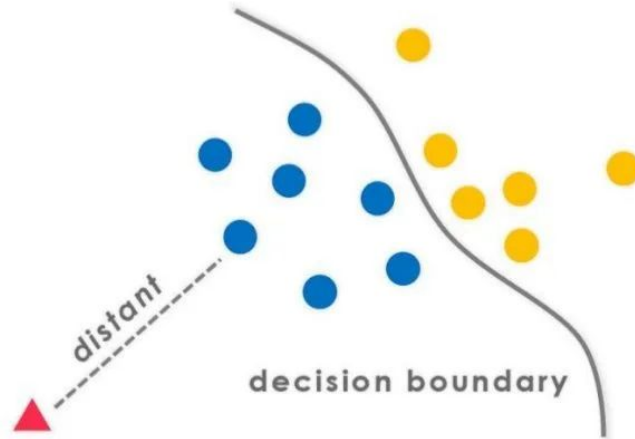
Interviewer: “You have 300 labeled support tickets and a demo on Friday. Which model do you train first, and why?”

- A) Naive Bayes - fast to fit, works with little data
- B) Logistic Regression - fewer assumptions, higher ceiling
- C) A neural network - it is 2026
- D) Naive Bayes now, logistic regression once tickets pile up

30 seconds, no talking. Then raise your hand for one letter.

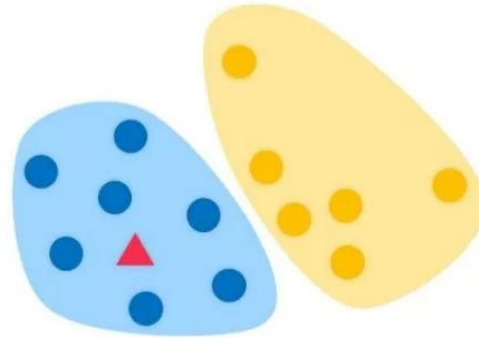
Discriminative vs. Generative

Discriminative



- Only care about estimating the conditional probabilities
- Very good when underlying distribution of data is really complicated (e.g. texts, images, movies)

Generative



- Model observations (x,y) first, then infer $p(y|x)$
- Good for missing variables, better diagnostics
- Easy to add prior knowledge about data



Naive Bayes (Generative)



Bayes' Rule

- Prediction is based on updating belief/class after seeing evidence

$$p(y|x) = \frac{p(x|y)p(y)}{p(x)}$$

- **Prior** $p(y)$: how common each class is
- **Likelihood** $p(x|y)$: how features look under each class
- **Posterior** $p(y|x)$: updated belief after seeing
- 💡 belief updating → start with a guess $p(y)$, refine with data $p(x|y)$



Naive Bayes Assumption

- Assuming features are **conditionally independent** given the class

$$p(x|y) = \prod_{j=1}^d p(x_j|y)$$

- Big simplification: turns complex joint distribution into a product
- Works surprisingly well for various applications
 -  each pixel $\approx$ independent feature
 -  word presence treated as independent
- Intuition: Given the class, every word casts its own vote 



Example

Suppose we want to **classify** emails as **Spam** ($y=1$) or **Ham** ($y=0$).

Vocabulary (features): {lottery, meeting, beef}

Thus each feature x_j stands for whether that word appears in the email (0/1)

👎 “Win a free lottery ticket” $\rightarrow x = [1, 0, 0]$

👍 “We will have a meeting” $\rightarrow x = [0, 1, 0]$

👍 “beef sometimes holds a meeting” $\rightarrow x = [0, 1, 1]$

🤔 “You have won a lottery!”



Bernoulli Naive Bayes (binary features)

- Features: binary word presence/absence (0 or 1)
- Each word = yes/no vote 
 - If present → contributes evidence
 - If absent → neutral or negative evidence
- Common for short text (SMS, tweets, headlines)
- Formula

$$P(x_j \mid y) = \theta_{j|y}^{x_j} (1 - \theta_{j|y})^{1-x_j}$$



Learning Parameters (MLE)

- Class prior estimated by class frequencies:

$$\hat{P}(y) = \frac{N_y}{N}$$

- Word probabilities estimated by frequency inside each class:

$$\hat{\theta}_{j|y} = P(x_j=1|y) = \frac{N_{j,y}}{N_y} = \frac{\text{Count of word } x_j \text{ in class } y}{\text{Count of doc in class } y}$$

Bernoulli Naive Bayes (binary features)



$$P(x_j = 1 \mid y) = \hat{\theta}_{j|y} \quad P(x_j = 0 \mid y) = 1 - \hat{\theta}_{j|y}$$

$$P(x_j \mid y) = \theta_{j|y}^{x_j} (1 - \theta_{j|y})^{1-x_j}$$

Mock Interview



Interviewer: “Here is the training set. A new email says ‘You have won a lottery!’, so $x = [1, 0, 0]$. What is $P(x \mid \text{Ham})$?”

- A) Small but positive
- B) Exactly 0
- C) 0.6, Ham is the more common class
- D) Cannot tell without more data

Word	Spam (N=4)	Ham (N=6)
lottery	3	0
meeting	1	5
beef	0	2

No talking. Then raise your hand for one letter.



Smoothing (Laplace / Add- α)

Problem: unseen word $\rightarrow$ zero probability

Solution: add α to counts

$$\hat{\theta}_{j|y} = \frac{N_{j,y} + \alpha}{N_y + \alpha c}$$

where c is number of classes

Effect: prevents zeros, makes model more robust



Numerical Stability



Multiplying many small probabilities → **underflow**

- Example: $0.001^{100} \approx 0$ on a computer

Solution: switch to log-space (turn products into sums)

$$\log P(y \mid x) \propto \log P(y) + \sum_{j=1}^d \log P(x_j \mid y)$$

Benefits:

- Avoids numerical underflow
- Computation becomes addition (faster, more stable)

PS: All modern NB implementations use log probabilities internally

Gaussian Naive Bayes (continuous features)



- Features: continuous values (e.g. numeric features)
- Assumption: each feature follows a Gaussian distribution within each class

$$P(x_j|y) = \frac{1}{\sqrt{2\pi\sigma_{j,y}^2}} \exp\left(-\frac{(x_j - \mu_{j,y})^2}{2\sigma_{j,y}^2}\right)$$



Practical Pipeline



Step 1 - Preprocessing

- Tokenize, lowercase, remove stopwords
- Handle missing values if needed



Step 2 - Feature Extraction

- Bag-of-Words / binary counts $\rightarrow$ Bernoulli NB
- Word counts / TF-IDF $\rightarrow$ Multinomial NB
- Continuous features $\rightarrow$ Gaussian NB



Practical Pipeline



Step 3 - Classifier

- Choose model: BernoulliNB, MultinomialNB, GaussianNB
- Learn parameters by frequency counts (MLE + smoothing)



Step 4 - Evaluation

- Metrics: Accuracy, Precision, Recall, Confusion Matrix
- Use `cross_val_score` for validation



When NB Works / Fails



Works well:

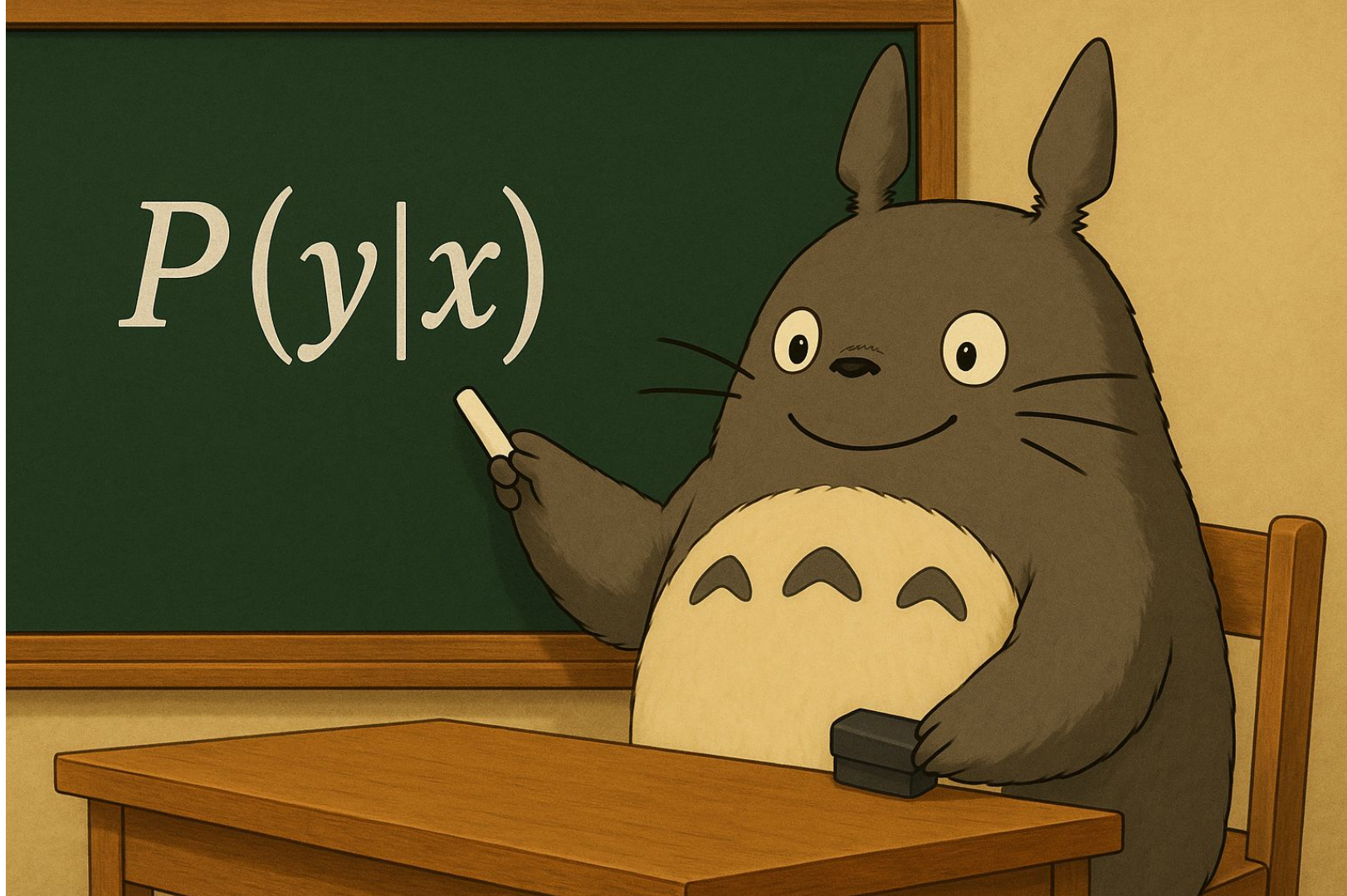
- High-dimensional sparse features (text, word counts)
- Small datasets (few samples per class)



Fails when:

- Strong feature correlations
- Complex dependencies between features

$$P(y|x)$$





Linear Regression



From Generative to Discriminative

- Naive Bayes: model joint distribution $P(x,y)$
- Linear Regression: directly fit $f(x) \approx y$

Do not learn the recipe, just draw the line that fits the taste 

Goal: **minimize classification error**, not model data generation



Hypothesis Form

Linear model = weighted sum of features

$$\hat{y} = \theta^\top x = \sum_{j=1}^d \theta_j x_j + b$$

Interpretation

- Each feature contributes **proportionally** to the prediction
- The weight tells us how much that feature **matters**

Trick

- add $x_0 = 1 \rightarrow$ bias term included
- Ensures model can fit data that does **not** pass through the origin



Loss Function (MSE)

Residual sum of squares (RSS):

$$L(\theta) = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \theta^\top x_i)^2$$

Intuition: penalizes more on **large errors**



Gradient Descent (Iterative Optimization)

Update rule

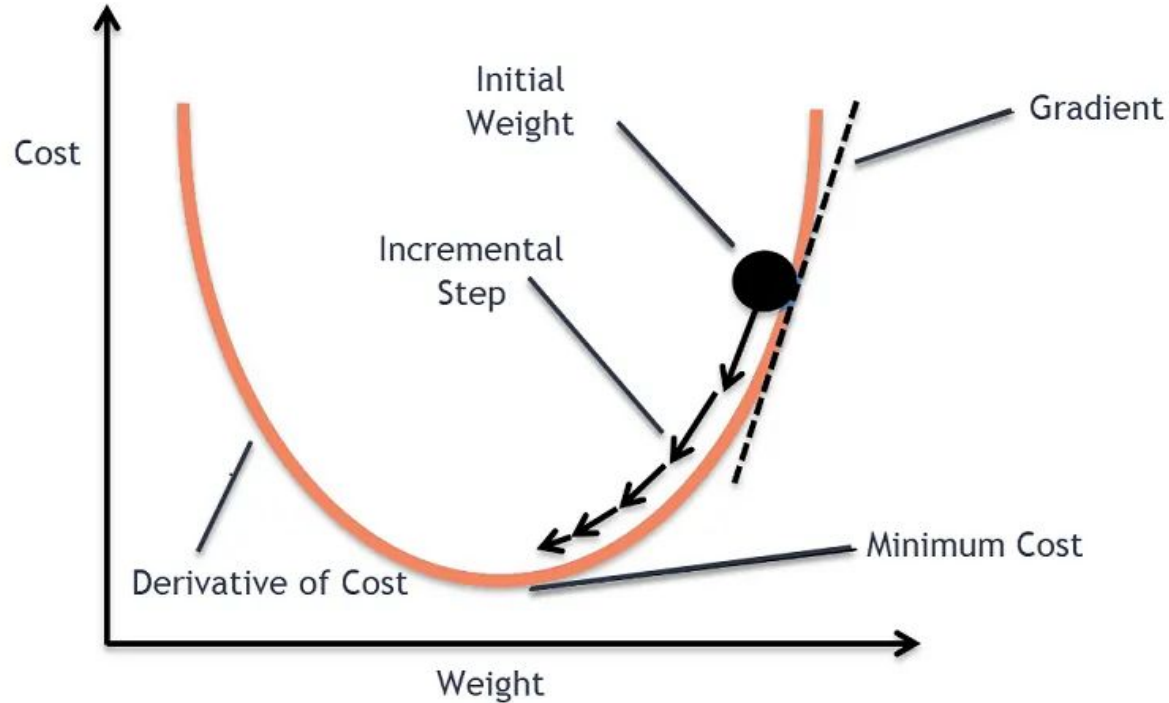
$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta)$$

where η = learning rate/step size

Works for large datasets & online learning

Cornerstone of DL 

Gradient Descent Intuition





Ordinary Least Squares (Closed Form)

Analytic solution when $X^T X$ invertible:

$$\theta^* = (X^T X)^{-1} X^T y$$

Analytic solution = one-shot computation
Fast for small/medium datasets

Limitation: unstable with collinearity or ill-conditioned



Regularization Motivation

Problems:

- Overfitting with too many features
 - Multicollinearity $\rightarrow$ unstable coefficients
-
- Solution: add **penalty** to shrink coefficients
 - Leads to **Ridge** & **Lasso regression**



Ridge Regression

Loss with L-2 penalty:

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - \theta^\top x_i)^2 + \alpha \|\theta\|_2^2$$

- Always unique solution
- Coefficients shrink towards zero (but not exactly zero)
- Good when many small/medium effects are expected



Lasso Regression

Loss with L-1 penalty:

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - \theta^\top x_i)^2 + \alpha \|\theta\|_1$$

- Encourages sparsity $\rightarrow$ some coefficients exactly 0
- Automatic feature selection
- Harder optimization (non-differentiable at 0)

Mock Interview



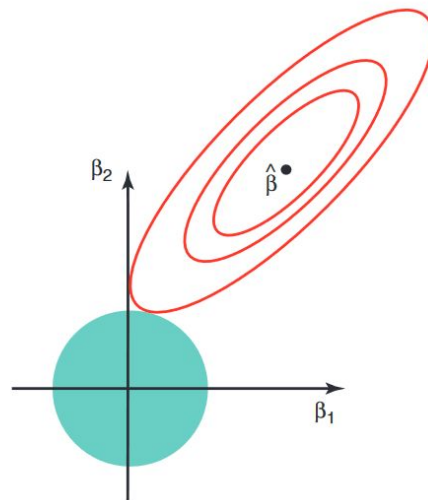
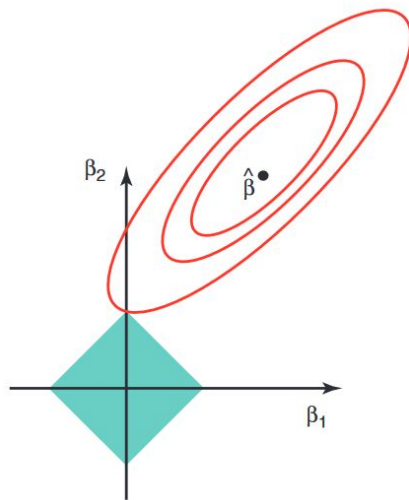
Interviewer: “10,000 features, you suspect only ~ 50 matter, and legal wants to see which ones drive the prediction. Ridge, Lasso, or plain OLS?”

- A) OLS - no penalty, no bias
 - B) Ridge - always a unique, stable solution
 - C) Lasso - drives most coefficients to exactly 0
 - D) Ridge first, then drop the smallest coefficients by hand
- 30 seconds, no talking. Then raise your hand for one letter.

Ridge vs Lasso (Geometry)



Lasso	Ridge
diamond constraint $\rightarrow$ corners	circular constraint $\rightarrow$ smooth shrinkage



Summary of Linear Regression



- Strengths: simple, interpretable, fast baseline
- Limitations: sensitive to outliers, assumes linearity
- Extensions: logistic regression, neural networks



NB vs. LR

Comparison



Aspect	Naive Bayes (Generative)	Linear Regression (Discriminative)
Model	Learns joint $P(x,y)$	Learns conditional $P(y x)$ directly
Philosophy	Recipe of data → simulate how data is generated	Boundary drawing → just separate outcomes
Features	Like word presence (independent assumption)	Fits weighted sum of features
Strengths	Simple, fast, good for text, small data	Interpretable, flexible, accurate with enough data
Limitations	Independence assumption unrealistic	Sensitive to outliers, needs more data



Case Study

Why "Simple" Math Still Powers the Internet's Revenue Engine



The Reality Check

- While LLMs (Generative) dominate the news, Linear Models (Discriminative) dominate the revenue
- Linear Regression & Logistic Regression handle >90% of real-time bidding decisions at the edge

Why Not Just Use Deep Learning? - The "Impossible" Triangle of AdTech

- **Latency**: Decisions must be made in < 10ms.
- **Scale**: Systems handle billions of requests per second (QPS).
- **Accuracy**: A 0.1% drop in prediction accuracy = Millions of dollars lost

The Verdict

- Deep Learning is for Understanding (User embedding, Image analysis)
- Linear Models are for Decision Making (Bidding, Budget Pacing, Calibration)



Survival of the Fastest: The Retrieval & Ranking Funnel

The 100ms Constraint

- From user request to ad display: Strict 100ms deadline
- Network overhead consumes ~40ms; Model Inference has < 20ms

The Ad Selection Funnel:

- Retrieval (Millions of Ads):
 - Model: Dot Product (Linear) or Vector Search
 - Goal: Fast, coarse filtering
- Rough Ranking (Thousands of Ads):
 - Model: Logistic Regression or Two-Tower Simple Models
 - Why: We cannot afford to run a Transformer on 10,000 candidates
- Precise Ranking (Top 10 Ads):
 - Model: Deep Neural Networks (DNNs)
 - Note: Most "Wide & Deep" architectures where the "Wide" part is a massive Linear Model to memorize specific feature interactions



Calibration is King: From Prediction to Probability

The Billion-Dollar Equation

In AdTech, we do NOT just classify (Click vs. No Click). We need an exact price:

$$\text{Bid} = \text{Value} \times \text{pCTR}$$

The Deep Learning Problem (Overconfidence)

Deep Neural Networks (DNNs) focus on ranking order, not probability

Issue: They are often "uncalibrated"

The Solution: Logistic Regression

Industry Standard: We feed it into a simple Logistic Regression layer to "scale" it:

$$P(\text{click}) = \text{sigmoid}(w \cdot \text{DNN_score} + b)$$

- Why? Logistic Regression minimizes Log-Loss, which specifically optimizes for probabilistic accuracy, ensuring the prediction matches reality

Mock Interview



Interviewer: “Your new DNN beats logistic regression on AUC, but after launch the ad campaigns start losing money. What went wrong?”

- A) AUC was measured on the wrong split
- B) It ranks well but its probabilities are off - bids use pCTR, not rank
- C) Deep models are too slow for the 10ms budget
- D) Nothing - AUC is the metric, keep the DNN

30 seconds, no talking. Then raise your hand for one letter.

Level Up: What to Ask Your AI Tutor 🚀



To deepen your understanding of today's lecture, try these prompts:

On The Billion-Dollar Ad Model:

- Why is Regression still the industry standard for Ad pricing?
- Explain the concept of 'Probability Calibration' and why we must optimize for Log-Loss (not Accuracy) when real money is at stake.

On Real-Time System Design (<10ms):

- I need to build an ad server with a 10ms latency limit. Design a 'Two-Stage Cascade' where a Linear Model handles fast retrieval and a Deep Model handles precise ranking. Why is this hybrid approach necessary?